

Microcontrollers – Lab 3 Serial Interface (UART)

Name: Michael McCorkell

Student ID: N01500049

Course: Microcontrollers Winter 2026

Instructor: Nilufar Poshtiban

Date: 13th of March, 2026

Abstract

This research demonstrates how to build UART serial communication on the TM4C123 microcontroller. The experiment involved configuring UART0 to send and receive data between the development board and a PC terminal program using the standard 8-N-1 serial settings. Several applications have been developed to demonstrate continuous transmission, character reception, echo capabilities, LED control by written commands, and string transmission and reception. The device successfully displayed submitted text in the terminal and responded to user interactions via both terminal output and onboard RGB LED activity. These results show that UART may be used efficiently to enable asynchronous communication between embedded devices and a computer. The lab also covered register-level configuration, baud rate selection, and the relationship between delivered data and observable hardware reaction.

Introduction

The purpose of this project was to set up UART0 on the TM4C123 microcontroller and utilize it to transfer serial data to a personal computer. The work required delivering text to a terminal, receiving typed characters from the terminal, displaying or interpreting received data, and developing reusable UART routines for string-based communication.

UART (Universal Asynchronous Receiver Transmitter) is a common serial communication mechanism in embedded systems. Because it is asynchronous, both devices must have the same communication parameters, including baud rate, number of data bits, parity, and stop bits. In this experiment, the UART interface was configured at 9600 baud, with 8 data bits, no parity, and one stop bit. These settings make it simple and reliable to send ASCII characters between the microcontroller and a terminal program such as PuTTY. The project also demonstrated how UART input may be linked to real hardware outputs by controlling the onboard RGB LEDs in response to received commands.

Objective

The goal of this lab was to use UART to establish serial communication between a TM4C123 microcontroller and a PC. The lab tasks included sending data from the microcontroller to the computer, receiving characters from a terminal software, and reacting to human requests.

Equipment

- TI TM4C123 Tiva C LaunchPad
- USB cable / USB-to-Serial interface
- Computer with Keil IDE
- PuTTY terminal software
- Connecting wires

UART Configuration

The UART interface was configured using 9600 baud rate, 8 data bits, no parity, and 1 stop bit (8-N-1 configuration).

Circuit Setup

UART0 was used for communication. Pin PA0 was configured as UART0 RX and pin PA1 was configured as UART0 TX. These pins were connected to the PC through a USB-to-Serial interface.

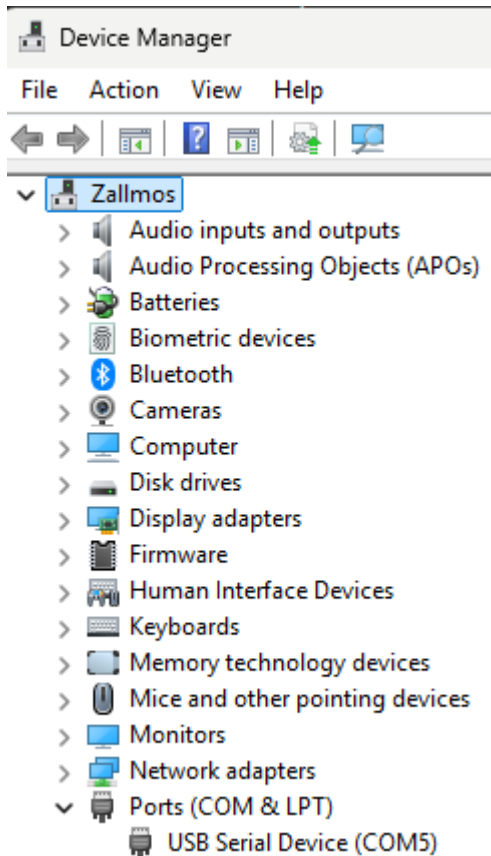


Figure 1: Device Manager Screenshot

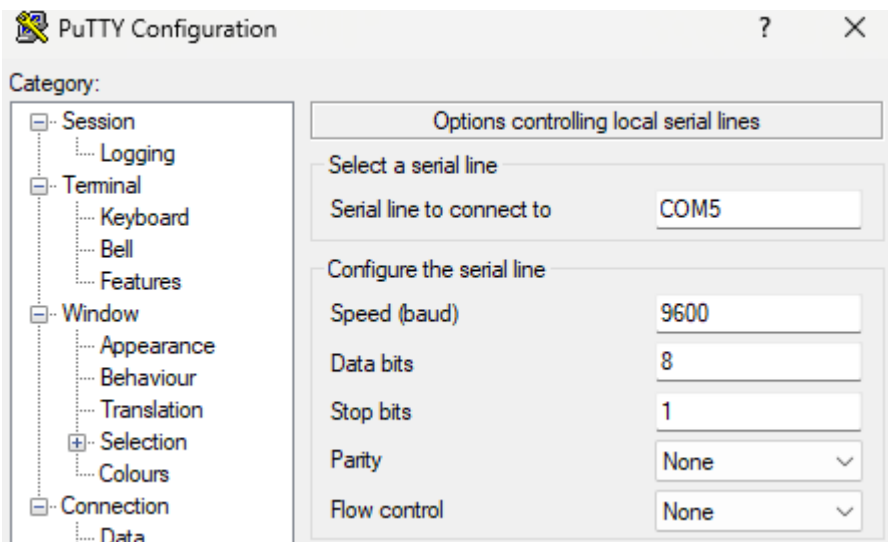


Figure 2: PuTTY Configuration Screenshot

Methods and Materials

The experiment used the TM4C123 microcontroller board, the Visual Studio IDE, and serial terminal software (PuTTY). UART0 and GPIO Port A were activated via the system control registers. PA0 and PA1 were switched to alternate function mode to allow UART communication. The UART line control register was configured for 8-bit transmission without parity or FIFO, and the baud-rate divisor registers were set to the proper connection rates. During testing, the PC terminal was used to send and receive serial data, while the onboard LEDs were checked for output replies.

Experimental Procedure

First, the board was connected to the PC, and the appropriate COM port was located in Device Manager. The terminal software was then set to 9600 baud with 8 data bits, no parity, one stop bit, and no flow control. In Activity 1, a transmission program was loaded to send the word YES to the terminal indefinitely, and the baud rate was then changed to 115200 for comparison. In Activity 2, a reception program was evaluated by entering text into the terminal and analyzing LED activity, echo output, and command-based replies. In Activities 3 and 4, the string transmission and string reception routines were developed and tested by passing complete messages between the terminal and the microcontroller.

Activity 1 – UART Transmission

In this activity the microcontroller continuously transmitted the message 'YES' to the terminal window.

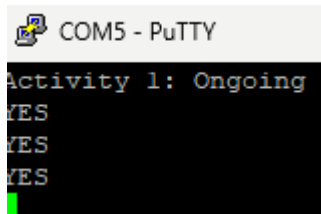
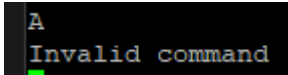
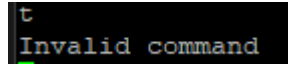


Figure 2: Activity 1 PuTTY Terminal Screenshot

Activity 2 – UART Reception and LED Control

The microcontroller received characters from the terminal. Commands were used to control the RGB LEDs and the received character was echoed back to the terminal.

Character	Observed LED Pattern
A	Red Light
A	 Invalid Command
t	Green Light
t	 Invalid Command
5	Yellow Light
5	Invalid Command

?

```
5
Invalid command
```

White Light

?

Invalid Command

```
?
Invalid command
```

```
Activity 2: Ongoing

Commands:
r -> Red LED
g -> Green LED
b -> Blue LED
o -> All off
h -> Print menu again
* -> Funny secret message
```

Figure 3: Activity 2 PuTTY Terminal Screenshot

Activity 3 – UART String Transmission

A function named UART0_Tx_String() was implemented to transmit full strings through UART.

```
static void UART0_Tx_String(char str[]){
    int i = 0;
    while(str[i] != '\0'){
        UART0_Tx_Char(str[i]);
        i++;
    }
}
```

Activity 4 – UART String Reception

A function named UART0_Rx_String() was implemented to receive a full string from the terminal until the Enter key was pressed.

```
Activity 4: Ongoing...
Enter a string: Hello World!
You typed: Hello World!
Activity 4: Completed...
```

Figure 4: Activity 4 PuTTY Terminal Screenshot

Results

UART connection was successfully established between the microcontroller and the PC terminal. The word YES was regularly sent over the terminal, typed letters were successfully accepted and echoed back, and the onboard RGB LEDs reacted to established command inputs. The string transmission and reception functionalities also worked well, passing complete messages between the terminal and the microcontroller.

Discussion

The results show that the UART0 peripheral was appropriately setup, and that the send and receive operations worked as expected. Continuous transmission in Activity 1 confirmed that the UART data channel between the microcontroller and the terminal was operational. In Activity 2, the successful echo of received characters demonstrated that the board was capable of receiving and re-transmitting serial data. The LED command answers demonstrated that incoming characters may be used to control hardware outputs in real time, in addition to displaying them on the terminal.

Any communication errors in UART systems are commonly caused by mismatched baud rates, incorrect COM port selection, or improper terminal settings. In this lab, using matching serial settings on both the board and the terminal was essential for readable output. The experiment also demonstrated an important limitation of asynchronous serial communication: if timing parameters do not match, corrupted characters will appear. Overall, the experiment successfully illustrated how register-level UART configuration supports reliable data exchange in embedded systems.

Lab Questions

1. Why use 8-N-1 configuration?

Eight data bits allow transmission of ASCII characters, no parity simplifies communication, and one stop bit ensures synchronization.

2. What happens if baud rates do not match?

If the baud rates differ, the receiver samples bits at incorrect times and data becomes corrupted.

3. How was correct communication verified?

Correct operation was verified by observing transmitted messages in the terminal and confirming that typed characters were echoed back.

4. COM Port Identification

USB Serial Device (COM5), you can also view Figure 1, for visual proof.

5. Code lines enabling UART

```
SYSCCTL->RCGCUART |= 1; (enable UART clock)
UART0->CTL = 0x301; (enable UART, TX, and RX)
```

6. Disconnecting and reconnecting the board

When the TM4C123 board was disconnected while PuTTY was still open, PuTTY displayed the error **“PuTTY Fatal Error: Error reading from serial device.”** This happens because the USB-to-serial connection is interrupted and the terminal can no longer communicate with the COM port.

When the board is reconnected, the microcontroller restarts because power is restored through the USB connection. However, PuTTY may need to be restarted to re-establish proper communication with the serial port. In some cases, the board also needs to be manually reset to ensure the previously programmed code runs correctly.

7. Transmission Time Calculation

Each UART frame contains **1 start bit, 8 data bits, and 1 stop bit**, for a total of **10 bits per character**.

For 100 characters:

$$100 \times 10 = 1000 \text{ bits}$$

At **9600 baud**, the theoretical transmission time is:

$$1000 / 9600 = 0.104 \text{ seconds}$$

Therefore, the theoretical time required to transmit 100 characters is approximately **0.104 seconds (104 ms)**.

Measured Time

The transmission time was measured using a stopwatch while observing the characters being printed in the terminal. The measured time was approximately:

13 seconds

This measured value is significantly larger than the theoretical value because the measurement was done manually using human reaction time. The delay in starting and stopping the stopwatch, along with the time taken to visually observe the output in the terminal, introduced large errors in the measurement. Additionally, terminal display delays can further increase the apparent transmission time.

As a result, the theoretical calculation provides a much more accurate representation of the actual UART transmission speed.

Conclusion

This lab demonstrated UART communication between the TM4C123 microcontroller and a PC terminal. Data was successfully transmitted and received, and serial commands were used to control onboard LEDs. The experiment reinforced the importance of correct UART configuration, register-level programming, and terminal setup in embedded serial communication.

References

Writing advice home. Writing Advice. (n.d.). <https://advice.writing.utoronto.ca/types-of-writing/lab-report/>

Mazidi, M. A., Chen, S., Naimi, S., & Naimi, S. (2017). Ti arm peripherals programming and interfacing: Programming arm cortex-M4 TM4C123G WITH C. MicroDigitalEd.com.

OpenAI. (2023). ChatGPT (Feb 13 version) [Large language model]. <https://chat.openai.com> "To rewrite sentences in a clear understanding"

Paraphrasing tool (AD-free and no sign-up required) - quillbot AI. Paraphrasing Tool (Ad-Free and No Sign-up Required) - QuillBot AI. (n.d.). <https://quillbot.com/paraphrasing-tool>